

Τμήμα Πληροφορικής και Τηλεματικής
Χαροκόπειο Πανεπιστήμιο
ΠΜΣ Βαθιά Μάθηση

1η Άσκηση: Ανάπτυξη και Εκπαίδευση ενός Πλήρως Διασυνδεδεμένου Νευρωνικού Δικτύου

Έκδοση 1.1

Διδάσκων: Χρήστος Δίου
Επικουρική Διδασκαλία: Αριστοτέλης Μπάλλας

1 Εισαγωγή

Η άσκηση αυτή έχει ως σκοπό την εξοικείωση με τους αλγορίθμους εκπαίδευσης των νευρωνικών δικτύων. Η εργασία αποσκοπεί στην κατανόηση και εφαρμογή του αλγορίθμου `backpropagation`, τόσο θεωρητικά (μέσω χειροκίνητων υπολογισμών) όσο και πρακτικά (μέσω υλοποίησης σε Python με χρήση της βιβλιοθήκης `NumPy`).

Παραδοτέα

Για την υποβολή της εργασίας, θα χρειαστεί να υποβληθούν, σε ένα zip file

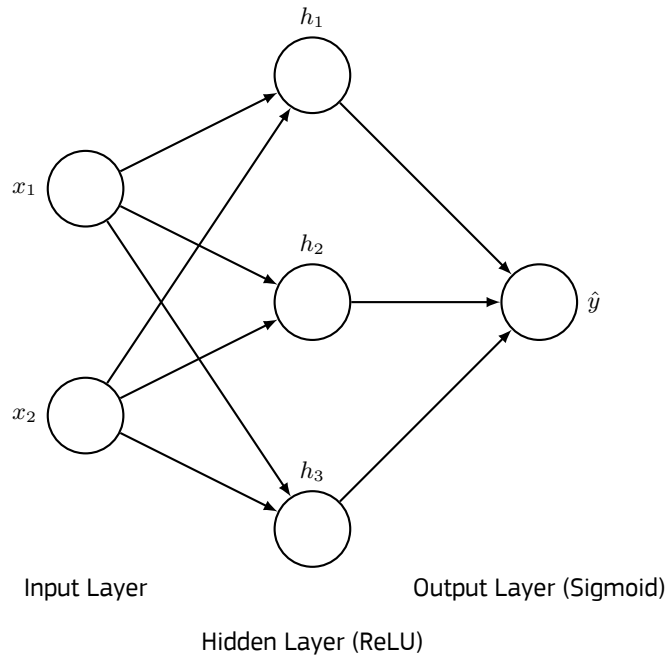
- Ένα αρχείο `fully_connected_nn.py` καθώς και 3 αρχεία `experiment1.py`, `experiment2.py` και `experiment3.py` με τον κώδικα σε γλώσσα python, με επαρκή σχόλια και επεξηγήσεις όπου θεωρείτε ότι χρειάζεται. Ο κώδικας θα πρέπει να μπορεί να εκτελεστεί σε περιβάλλον python3 με πρόσφατες εκδόσεις των βιβλιοθηκών `numpy` και `torch`.
- Μια αναφορά που να φαίνονται αναλυτικά οι εξισώσεις και υπολογισμοί σας κατά την εφαρμογή του `backpropagation`, να περιγράφει τα αποτελέσματα της εκτέλεσης του κώδικά σας, καθώς και απαντήσεις στα ερωτήματα της εργασίας. Η αναφορά μπορεί να περιέχει και τυχόν λεπτομέρειες για τον τρόπο εκτέλεσης του κώδικα, αν το κρίνετε απαραίτητο.

Την άσκηση θα πρέπει να την υλοποιήσετε ατομικά.

2 Εφαρμογή του αλγορίθμου Backpropagation

Το πρώτο μέρος της εργασίας εστιάζει στον αναλυτικό υπολογισμό των forward pass και backpropagation, σε ένα πλήρως διασυνδεδεμένο Νευρωνικό Δίκτυο. Το δίκτυο αυτό θα αποτελείται από:

- 2 νευρώνες εισόδου
- 3 νευρώνες στο κρυφό επίπεδο (με ReLU ενεργοποίηση)
- 1 νευρώνα εξόδου (με Sigmoid ενεργοποίηση)
- Συνάρτηση απώλειας: Δυαδική Διεντροπία (Binary Cross-Entropy (BCE))
- Ρυθμό εκμάθησης $\eta = 0.1$



Σχήμα 1: Πλήρως διασυνδεδεμένο νευρωνικό δίκτυο με ένα κρυφό επίπεδο.

Το δίκτυο είναι αυτό της Εικόνας 1, και θεωρούμε τα εξής:

Είσοδος:

$$X = \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix}$$

Αναμενόμενη Έξοδος:

$$Y = 1$$

Αρχικοποίηση των βαρών και μεροληψιών:

$$W_1 = \begin{bmatrix} 0.1 & 0.4 \\ 0.2 & 0.5 \\ 0.3 & 0.6 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix}$$

$$W_2 = [0.2 \quad 0.3 \quad 0.4], \quad b_2 = [0.1]$$

Ζητούμενα:

1. Υπολογίστε το **Forward Pass** (ενεργοποιήσεις Z_1, A_1, Z_2, A_2).
2. Υπολογίστε την **απώλεια** με τη συνάρτηση δυαδικής διεντροπίας (BCE).
3. Υπολογίστε τις **παραγώγους** (gradients) των βαρών και των μεροληψιών μέσω **backpropagation**.
4. Ενημερώστε τις **παραμέτρους** χρησιμοποιώντας τον αλγόριθμο **Gradient Descent**.

Σημειώσεις:

- Παρουσιάστε όλα τα ενδιάμεσα βήματα και τους υπολογισμούς σας.
- Η παράγωγος της σιγμοειδούς συνάρτησης $\sigma(x) = \frac{1}{1+e^{-x}}$ υπολογίζεται ως $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ ενώ της ReLU $ReLU(x) = \max(0, x)$ ως $ReLU'(x) = 1$, αν $x > 0$ και $ReLU'(x) = 0$, αν $x \leq 0$
- Ως σύμβαση για τα βάρη, θεωρούμε ότι το $w_{jk}^{(l)}$ αντιστοιχεί στο βάρος που συνδέει τον νευρώνα k του επιπέδου $(l - 1)$ με τον νευρώνα j του επιπέδου l . Ακόμα, $a_j^{(l)} = f(z_j) = f(\sum_k w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)})$ είναι η έξοδος ενός νευρώνα.

3 Υλοποίηση πλήρως διασυνδεδεμένου νευρωνικού δικτύου με Numpy

Δημιουργήστε το αρχείο `fully_connected_nn.py` το οποίο θα υλοποιεί την κλάση

```
FullyConnectedNN
```

Η κλάση αυτή υλοποιεί ένα πλήρως διασυνδεδεμένο νευρωνικό δίκτυο. Για την υλοποίηση πρέπει να χρησιμοποιήσετε τη βιβλιοθήκη `numpy` και `matplotlib` της python και *καμία άλλη*.

Σημείωση: Προσπαθήστε όσο μπορείτε να υλοποιήσετε τις παρακάτω μεθόδους χρησιμοποιώντας πράξεις μεταξύ πινάκων και διανυσμάτων. Αυτό θα επιταχύνει πολύ σημαντικά το χρόνο εκτέλεσης του κώδικά σας, καθώς θα αξιοποιεί τη βελτιστοποιημένη υλοποίηση της `numpy` για πράξεις γραμμικής άλγεβρας.

3.1 Κλάση και ιδιότητες

Η `FullyConnectedNN` έχει τις εξής ιδιότητες (attributes)

- `input_size`: Αριθμός νευρώνων στο επίπεδο εισόδου, ο οποίος αντιστοιχεί στα χαρακτηριστικά των δεδομένων
- `hidden_size`: Αριθμός που αντιστοιχεί στους νευρώνες του κρυφού επιπέδου μοντέλου
- `output_size`: Αριθμός νευρώνων στο επίπεδο εξόδου
- `lr`: Ρυθμός εκμάθησης για την εκπαίδευση του μοντέλου

Θα χρειαστεί να προσθέσετε και άλλες ιδιότητες στο μοντέλο σας, όπως το πλήθος των δεδομένων N με τα οποία εκπαιδεύτηκε, ή και προσωρινές μεταβλητές, όπως οι παράγωγοι των w και b σε κάθε βήμα βελτιστοποίησης, καθώς και η πρόβλεψη $p(1|x)$ για τα δεδομένα εισόδου x κατά την εκπαίδευση (δείτε τις επόμενες ενότητες). Οι ιδιότητες της κλάσης μπορούν να αρχικοποιούνται με `None` (στην `__init__()`), εκτός από το ρυθμό εκμάθησης. Αυτός πρέπει να μπορεί να δοθεί ως είσοδος στην `__init__()`, με προεπιλεγμένη τιμή 10^{-2} .

Οι μέθοδοι της `FullyConnectedNN` περιγράφονται στα ακόλουθα.

3.2 `init_parameters(self)`

Η μέθοδος αρχικοποιεί παραμέτρους $W^{(l)}$ και $b^{(l)}$ για κάθε επίπεδο, με δειγματοληψία από Γκαουσιανή κατανομή με μέση τιμή $\mu = 0$ και τυπική απόκλιση $\sigma = 0.1$. Αυτό μπορείτε να το πετύχετε με τη χρήση της συνάρτησης `random.randn()`.

3.3 `forward(self, X)`

Δέχεται ως είσοδο έναν $N \times p$ πίνακα X , όπου p τα χαρακτηριστικά (features) των δεδομένων, κάθε γραμμή του οποίου αντιστοιχεί σε ένα δείγμα δεδομένων και υπολογίζει τις εξόδους των επιπέδων (Z_1, A_1, Z_2, A_2) σε μεταβλητές της κλάσης (πχ. `self.Z1` κοκ).

3.4 `predict(self, X)`

Επιστρέφει τις τελικές προβλέψεις του μοντέλου. Θεωρήστε κατώφλι απόφασης για τη δυαδική κατηγοριοποίηση το 0.5.

3.5 `loss(self, X, y)`

Υπολογίζει την τιμή της συνάρτησης απώλειας, δηλ. τη δυαδική διεντροπία του μοντέλου για τα δεδομένα X, y .

3.6 backward(self, X, y)

Θα υπολογίζει τις παραγώγους των παραμέτρων $W^{(l)}$ και b^l του μοντέλου. Οι υπολογισμένες παράγωγοι να αποθηκευτούν σε μεταβλητή της κλάσης `self.gradients` ως ένα dictionary με κλειδιά τα (dW1, db1, dW2, db2).

3.7 step()

Η συνάρτηση εκτελεί ένα βήμα του αλγόριθμου Gradient Descent, δηλ.

$$w_i \leftarrow w_i - \eta \frac{\partial \mathcal{L}}{\partial w_i}$$

και αντίστοιχα για το b .

3.8 fit()

Η πλήρης δήλωση της συνάρτησης είναι

```
fit(self, X, y, iterations=10000, batch_size=None, show_step=1000)
```

Η `fit` δέχεται στην είσοδο έναν $N \times p$ πίνακα X , κάθε γραμμή του οποίου αντιστοιχεί σε ένα διάνυσμα χαρακτηριστικών του συνόλου εκπαίδευσης, και ένα $N \times 1$ διάνυσμα τιμών εξόδου y . Ο πίνακας και τα διανύσματα να είναι numpy arrays. Η συνάρτηση πρέπει να κάνει τα ακόλουθα:

- Να επιβεβαιώνει ότι τα X και y είναι numpy arrays και ότι οι μεταξύ τους διαστάσεις είναι συμβατές. Σε διαφορετική περίπτωση πρέπει να ενεργοποιεί `ValueError` exception (ή κάποιο άλλο αντίστοιχο)
- Να αρχικοποιεί τις παραμέτρους του μοντέλου καλώντας τη συνάρτηση `init_parameters()`
- Να τοποθετεί τα δείγματα (γραμμές) στον X σε τυχαία σειρά

Έπειτα πρέπει να ξεκινάει την επανάληψη εκπαίδευσης, όπου

- Επιλέγονται τα επόμενα `batch_size` δείγματα. Αν έχουμε φτάσει στο τέλος του πίνακα ξεκινάμε πάλι από την αρχή (μπορεί το τελευταίο batch να έχει λιγότερα από `batch_size` δείγματα). Αν το `batch_size` είναι `None` τότε σε κάθε επανάληψη επιλέγονται όλα τα δείγματα.
- Καλούνται διαδοχικά η `forward()` (υπολογίζονται οι τιμές των ενεργοποιήσεων και αποθηκεύονται στις αντίστοιχες μεταβλητές), η `backward()` (υπολογίζει τις παραγώγους) και η `step()` (εκτελεί ένα βήμα του gradient descent).
- Αν έχουν εκτελεστεί `show_step` βήματα, τότε καλεί τη `loss` για όλα τα δείγματα του συνόλου εκπαίδευσης και εκτυπώνει την τρέχουσα επανάληψη (πχ 1000) καθώς και την τρέχουσα τιμή του `loss`.

Η διαδικασία αυτή επαναλαμβάνεται για `iterations` επαναλήψεις. Η `fit` δεν επιστρέφει κάτι.

Το αρχείο `fully_connected_nn.py` και την κλάση που περιέχει θα το χρησιμοποιήσετε στα επόμενα πειράματα.

4 Πείραμα 1: Συνθετικά δεδομένα (Blobs & Flower)

Από αυτό το σημείο και έπειτα μπορείτε να χρησιμοποιήσετε και ορισμένες συναρτήσεις και κλάσεις της βιβλιοθήκης `scikit-learn`. Χρησιμοποιήστε το αρχείο `generate_datasets.py` που σας δίνεται

μαζί με την εκφώνηση της εργασίας. Καλέστε την `generate_binary_problem` για να παράξετε $N = 1000$ σημεία με κέντρα στο $(0, 0)$ και $(8, 8)$, δηλ. κέντρα που δίνονται από τον πίνακα

$$C = \begin{bmatrix} 0 & 8 \\ 0 & 8 \end{bmatrix} \quad (1)$$

Χωρίστε το σύνολο δεδομένων σε 70% σύνολο εκπαίδευσης και 30% σύνολο δοκιμής. Αρχικοποιήστε ένα αντικείμενο `FullyConnectedNN` και καλέστε την `fit` για τα δεδομένα εκπαίδευσης με παραμέτρους και `batch_size = None`. Τι παρατηρείτε για την τιμή της συνάρτησης απώλειας, καθώς προχωράει η εκπαίδευση του μοντέλου; Υπολογίστε και την ευστοχία του μοντέλου μετά την εκπαίδευση.

Από το αρχείο `helpers.py`, που σας δίνεται μαζί με την εκφώνηση της εργασίας, καλέστε την `plot_decision_boundary` και οπτικοποιήστε το όριο απόφασης του εκπαιδευμένου μοντέλου, χρησιμοποιώντας την κατάλληλη μέθοδο ως όρισμα. Τι παρατηρείτε;

Δοκιμάστε και γράψτε τις παρατηρήσεις σας και για άλλα, διαφορετικά κέντρα των κλάσεων, που αντιστοιχούν σε πιο δύσκολα διαχωρίσιμα προβλήματα.

Επαναλάβετε τα παραπάνω για ένα πιο δύσκολο διαχωρίσιμο πρόβλημα, φτιάχνοντας ένα σύνολο εκπαίδευσης μέσω της `generate_flower_problem` από το αρχείο `generate_datasets.py`. Τι παρατηρείτε; Πώς θα μπορούσε να λυθεί το συγκεκριμένο πρόβλημα και τί θα αλλάζατε;

Αποθηκεύστε τον κώδικά σας στο `experiment1.py`.

5 Πείραμα 2: Breast cancer dataset

Φορτώστε το σύνολο δεδομένων Breast Cancer του scikit-learn (δείτε το σχετικό notebook). Εφαρμόστε τα ακόλουθα βήματα για 20 επαναλήψεις:

- Χωρίστε το σύνολο δεδομένων σε 70% δεδομένα εκπαίδευσης και 30% δοκιμής.
- Κανονικοποιήστε τα δεδομένα στο $[0, 1]$
- Εκπαιδεύστε το δίκτυο που υλοποιήσατε παραπάνω με ρυθμό εκμάθησης 0.01 και `batch_size = 64`
- Αξιολογήστε την ευστοχία στο σύνολο δοκιμής

Υπολογίστε και σημειώστε το μέσο όρο και την τυπική απόκλιση της ευστοχίας του μοντέλου για τις 20 επαναλήψεις. Πόσο χρόνο κάνει να τρέξει αυτό το πείραμα στον υπολογιστή σας; Αναφέρετε τον επεξεργαστή, τη μνήμη του υπολογιστή σας, καθώς και το χρόνο εκτέλεσης σε δευτερόλεπτα.

Σημείωση: Προσοχή στις διαστάσεις του πίνακα του συνόλου εκπαίδευσης. Εάν έχει γίνει σωστή υλοποίηση της κλάσης του νευρωνικού, το μόνο που θα χρειαστεί να αλλάξετε είναι το μέγεθος της μεταβλητής `input_size` κατά την αρχικοποίηση του μοντέλου. Το ίδιο ισχύει και για το μέγεθος του κρυφού επιπέδου όπου μπορείτε να πειραματιστείτε με διαφορετικό αριθμό νευρώνων.

6 Πείραμα 3: Breast cancer dataset (PyTorch)

Κάντε ακριβώς ό,τι κάνατε στο προηγούμενο πείραμα, αλλά κατασκευάζοντας ένα μοντέλο `PyTorch`, χρησιμοποιώντας όλες τις βοηθητικές βιβλιοθήκες. Αναφέρετε και πάλι το χρόνο εκτέλεσης. Πως συγκρίνονται τα δύο μοντέλα, σε ό,τι αφορά την επίδοση και την αποτελεσματικότητά τους;